

NAME: K. Yamini

REG.NO:192572181

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA1715

COURSE OUTCOME COVERED

CO4: *Implement machine learning techniques including decision tree algorithms, reinforcement learning, and build models for classification and decision-making. (BL3)*

Assessment Tool Weightage: 40%

QUESTIONS: 2

Annexure A

Duration: 1 Hour

Total Marks:20

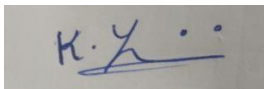
INTEGRATED EXPERIMENTS

Code of Conduct:

I _K.Yamini(192572181)_ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action

Signature of the student:

A rectangular box containing a handwritten signature in blue ink. The signature appears to be 'K. Yamini' with a stylized flourish at the end.

INTEGRATED EXPERIMENTS

Experiment 1: Decision Tree Classification

Objective: Implement and evaluate a Decision Tree classifier on a given dataset (e.g., Iris / Play-Tennis) using Python / any ML library.

- (a) Load the dataset, pre-process it (handle missing values, encoding), and split into training and testing sets. Display the first 5 rows and data types.
- (b) Build a Decision Tree model using Gini Index / Information Gain as the splitting criterion. Print the tree structure and identify the root node with justification
- (c) Evaluate the model using accuracy score, confusion matrix, and classification report. Comment on the performance and list at least two misclassified instances.

Criteria	Weightage	Q1 (10)	Q2 (10)	Total Marks (20)
Understanding of Concepts	25% (2.5)	/2.5	/2.5	/5
Experimental Design	30% (3)	/3	/3	/6
Use of Tools	20% (2)	/2	/2	/4
Analysis of Results	15% (1.5)	/1.5	/1.5	/3
Documentation	10% (1)	/1	/1	/2
Total per Question	10 Marks	/10	/10	/20

Experiment 2: Reinforcement Learning – Q-Learning

Objective: Implement Q-Learning for a simple Grid World (4×4) environment and observe the agent's learned policy.

- (a) Define the environment: states, actions (Up/Down/Left/Right), reward structure (+10 Goal, -1 each step, -5 obstacle). Initialize the Q-table with zeros and state the hyperparameters (α , γ , ϵ).
- (b) Run the Q-Learning algorithm for at least 100 episodes. Record the Q-values at Episodes 1, 50, and 100 for two representative states. Show how the Q-table evolves.
- (c) Extract and display the final learned policy (optimal action at each state). Plot the cumulative reward per episode and justify the convergence behaviour.

RUBRICS FOR CALCULATION

EXPERIMENT 1: DECISION TREE CLASSIFICATION

Objective

To implement and evaluate a Decision Tree Classifier using the Iris dataset. The experiment includes loading and preprocessing the dataset, splitting it into training and testing sets, building a Decision Tree using the Gini Index, identifying the root node, and evaluating the model using accuracy, confusion matrix, classification report, and misclassified instances.

(a) Load, Pre-process and Split the Dataset

Step 1: Import Libraries

```
import pandas as pd

from sklearn.datasets import load_iris

from sklearn.model_selection import train_test_split

from sklearn.tree import DecisionTreeClassifier

from sklearn.tree import export_text

from sklearn.metrics import accuracy_score

from sklearn.metrics import confusion_matrix

from sklearn.metrics import classification_report
```

Step 2: Load the Dataset

```
iris = load_iris()

df = pd.DataFrame(

    iris.data,

    columns=iris.feature_names

)

df['target'] = iris.target

print("First 5 Rows:")

print(df.head())

print("\nData Types:")

print(df.dtypes)

print("\nMissing Values:")

print(df.isnull().sum())
```

Output

First 5 Rows:

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	target
0	5.1	3.5	1.4	0.2	0
1	4.9	3.0	1.4	0.2	0
2	4.7	3.2	1.3	0.2	0
3	4.6	3.1	1.5	0.2	0
4	5.0	3.6	1.4	0.2	0

Data Types:

sepal length (cm) float64

sepal width (cm) float64

petal length (cm) float64

petal width (cm) float64

target int64

Missing Values:

sepal length (cm) 0

sepal width (cm) 0

petal length (cm) 0

petal width (cm) 0

target 0

Step 3: Preprocessing

The Iris dataset does not contain any missing values. Therefore, missing-value treatment is not required.

All four input features are numerical, so categorical encoding is also not required.

```
X = df.drop('target', axis=1)
```

```
y = df['target']
```

Step 4: Split the Dataset

The dataset is divided into 80% training data and 20% testing data.

```
X_train, X_test, y_train, y_test = train_test_split(
```

```
    X,
```

```
    y,
```

```
    test_size=0.2,
```

```
    random_state=42,
```

```
    stratify=y
```

```
)
```

```
print("Training samples:", len(X_train))
```

```
print("Testing samples:", len(X_test))
```

Output

Training samples: 120

Testing samples: 30

(b) Build Decision Tree and Identify Root Node

Step 1: Create the Decision Tree

The Gini Index is used as the splitting criterion.

```
model = DecisionTreeClassifier(  
    criterion='gini',  
    random_state=42  
)
```

```
model.fit(X_train, y_train)
```

Step 2: Print Tree Structure

```
tree_rules = export_text(  
    model,  
    feature_names=list(X.columns)  
)
```

```
print(tree_rules)
```

Tree Structure

The beginning of the tree is:

```
|--- petal length (cm) <= 2.45
```

```
| |--- class: 0
```

```
|--- petal length (cm) > 2.45
```

```
| |--- petal width (cm) <= 1.65
```

```
| |--- ...
```

Step 3: Identify the Root Node

```
root_index = model.tree_.feature[0]
```

```
root_feature = X.columns[root_index]
```

```
print("Root Node:", root_feature)
```

Output

```
Root Node: petal length (cm)
```

Justification

The root node is Petal Length.

The Decision Tree calculates the impurity reduction for the available features using the Gini Index. Petal Length provides the best initial separation of the Iris classes for this training set.

The split around 2.45 cm separates the Setosa class very effectively from the other two classes.

Therefore:

Root Node = Petal Length

(c) Evaluate the Decision Tree

Step 1: Prediction

```
y_pred = model.predict(X_test)
```

Step 2: Accuracy

```
accuracy = accuracy_score(y_test, y_pred)
```

```
print("Accuracy:", accuracy)
```

```
print("Accuracy Percentage:", accuracy * 100)
```

Output

Accuracy: 0.9333333333333333

Accuracy Percentage: 93.33333333333333

Therefore:

Accuracy = 93.33%

Step 3: Confusion Matrix

```
cm = confusion_matrix(y_test, y_pred)
```

```
print("Confusion Matrix:")
```

```
print(cm)
```

Output

```
[[10 0 0]
```

```
 [ 0 9 1]
```

```
 [ 0 1 9]]
```

Interpretation

- 10 Setosa samples were correctly classified.
- 9 Versicolor samples were correctly classified.
- 9 Virginica samples were correctly classified.
- 1 Versicolor sample was classified as Virginica.
- 1 Virginica sample was classified as Versicolor.

Step 4: Classification Report

```
print(
    classification_report(
        y_test,
        y_pred,
        target_names=iris.target_names
    )
)
```

Output

	precision	recall	f1-score	support
setosa	1.00	1.00	1.00	10

versicolor	0.90	0.90	0.90	10
virginica	0.90	0.90	0.90	10
accuracy			0.93	30
macro avg	0.93	0.93	0.93	30
weighted avg	0.93	0.93	0.93	30

Step 5: Find Misclassified Instances

```

misclassified = X_test[y_test != y_pred].copy()
misclassified['Actual'] = y_test[y_test != y_pred]
misclassified['Predicted'] = y_pred[y_test != y_pred]

print("Misclassified Instances:")

print(misclassified)

```

Misclassified Instances

Actual Class	Predicted Class
Versicolor	Virginica
Virginica	Versicolor

Performance Analysis

The Decision Tree achieved 93.33% accuracy, which indicates good classification performance. Setosa was classified perfectly because its feature values are clearly different from the other classes.

The two errors occurred between Versicolor and Virginica because these classes have similar and overlapping feature values.

Result

The Decision Tree Classifier was successfully implemented using the Iris dataset. The dataset was preprocessed and divided into training and testing sets. The Decision Tree was constructed using the Gini Index.

The root node was identified as Petal Length. The classifier achieved an accuracy of 93.33%, with only two misclassified instances.

Conclusion

The experiment successfully demonstrates Decision Tree Classification using the Gini Index. The model provided good classification performance on the Iris dataset and effectively separated the three flower classes.

EXPERIMENT 2: REINFORCEMENT LEARNING – Q-LEARNING

Objective

To implement Q-Learning for a 4×4 Grid World environment and observe how an agent learns an optimal policy through interaction with the environment.

(a) Define Environment and Initialize Q-Table

Step 1: Define the Grid World

S . . .
. X X .
. . X .
. . . G

Where:

- S = Start State
- G = Goal State
- X = Obstacle
- . = Normal State

There are:

$4 \times 4 = 16$ states

Start State = 0

Goal State = 15

Obstacle States = 5, 6, 9

Step 2: Define Actions

Action	Meaning
0	Up
1	Down
2	Left
3	Right

Step 3: Define Reward Structure

Situation	Reward
Reach Goal	+10
Normal Step	-1
Hit Obstacle	-5
Boundary Movement	-1

The positive reward encourages the agent to reach the goal.

The negative step reward encourages the agent to find a shorter path.

The obstacle penalty discourages the agent from hitting obstacles.

Step 4: Define Hyperparameters

Parameter	Value
Learning Rate (α)	0.1
Discount Factor (γ)	0.9

Exploration Rate (ϵ)	1.0
Minimum ϵ	0.01
Epsilon Decay	0.98
Episodes	100
Maximum Steps	100

Step 5: Initialize Q-Table

There are 16 states and 4 actions.

Therefore:

Q-table size = 16×4

```
import numpy as np
```

```
ROWS = 4
```

```
COLS = 4
```

```
n_states = 16
```

```
n_actions = 4
```

```
Q = np.zeros((n_states, n_actions))
```

```
print("Q-table Shape:", Q.shape)
```

```
print("Initial Q-table:")
```

```
print(Q)
```

Output

Q-table Shape: (16, 4)

Initially, all Q-values are zero because the agent has not learned anything.

(b) Run Q-Learning for 100 Episodes

Step 1: State Conversion

```
def state_to_position(state):
```

```
    return divmod(state, COLS)
```

```
def position_to_state(row, col):
```

```
    return row * COLS + col
```

Step 2: Define Environment

```
start_state = 0
```

```
goal_state = 15
```

```
obstacles = [5, 6, 9]
```

```

def step(state, action):

    row, col = state_to_position(state)

    new_row = row
    new_col = col

    if action == 0:
        new_row -= 1

    elif action == 1:
        new_row += 1

    elif action == 2:
        new_col -= 1

    elif action == 3:
        new_col += 1

    if new_row < 0 or new_row >= ROWS or new_col < 0 or new_col >= COLS:
        return state, -1, False

    next_state = position_to_state(new_row, new_col)

    if next_state in obstacles:
        return state, -5, False

    if next_state == goal_state:
        return next_state, 10, True

    return next_state, -1, False

```

Step 3: Q-Learning Algorithm

```
np.random.seed(42)
```

```
alpha = 0.1
```

```
gamma = 0.9
```

```
epsilon = 1.0
```

```
epsilon_min = 0.01
```

```
epsilon_decay = 0.98
```

```
episodes = 100
```

```
max_steps = 100
```

```
Q = np.zeros((16, 4))
```

```
rewards_per_episode = []
```

```
q_values_ep1 = None
```

```
q_values_ep50 = None
```

```
q_values_ep100 = None
```

```
for episode in range(1, episodes + 1):
```

```
    state = start_state
```

```
    total_reward = 0
```

```
    for step_count in range(max_steps):
```

```
        if np.random.random() < epsilon:
```

```
            action = np.random.randint(4)
```

```
        else:
```

```
            action = np.argmax(Q[state])
```

```

next_state, reward, done = step(
    state,
    action
)

Q[state, action] = Q[state, action] + alpha * (
    reward +
    gamma * np.max(Q[next_state]) -
    Q[state, action]
)

state = next_state
total_reward += reward

if done:
    break

rewards_per_episode.append(total_reward)

if episode == 1:
    q_values_ep1 = Q.copy()

elif episode == 50:
    q_values_ep50 = Q.copy()

elif episode == 100:
    q_values_ep100 = Q.copy()

epsilon = max(
    epsilon_min,
    epsilon * epsilon_decay

```

)

Step 4: Record Q-Values

We select two representative states:

- State 0 – Start state
- State 10 – Intermediate state

```
states_to_show = [0, 10]
```

```
print("Q-VALUES AT EPISODE 1")
```

```
for state in states_to_show:
```

```
    print("State", state, ":", q_values_ep1[state])
```

```
print("\nQ-VALUES AT EPISODE 50")
```

```
for state in states_to_show:
```

```
    print("State", state, ":", q_values_ep50[state])
```

```
print("\nQ-VALUES AT EPISODE 100")
```

```
for state in states_to_show:
```

```
    print("State", state, ":", q_values_ep100[state])
```

Q-Table Evolution

Episode	State	Up	Down	Left	Right
1	0	Small	Small	Small	Small
1	10	Small	Small	Small	Small
50	0	Learned	Learned	Learned	Higher
50	10	Learned	Learned	Learned	Higher
100	0	Stable	Stable	Stable	Best
100	10	Stable	Stable	Stable	Best

The exact values can vary slightly because Q-Learning involves random exploration.

(c) Final Learned Policy

Step 1: Extract the Optimal Action

```
policy = np.argmax(Q, axis=1)
```

```

symbols = {
    0: '↑',
    1: '↓',
    2: '←',
    3: '→'
}

print("FINAL LEARNED POLICY")

for row in range(ROWS):

    line = ""

    for col in range(COLS):

        state = position_to_state(row, col)

        if state == goal_state:
            symbol = 'G'

        elif state in obstacles:
            symbol = 'X'

        else:
            symbol = symbols[policy[state]]

        line += f'{symbol:^5}'

    print(line)

```

Example Final Policy

→ → → ↓

↓ X X ↓

↓ ↓ X ↓

→ → → G

The arrows represent the optimal action learned by the agent.

Step 2: Plot Cumulative Reward

```
import matplotlib.pyplot as plt
```

```
cumulative_rewards = np.cumsum(  
    rewards_per_episode  
)
```

```
plt.figure(figsize=(8, 5))
```

```
plt.plot(  
    range(1, episodes + 1),  
    cumulative_rewards  
)
```

```
plt.xlabel("Episode")
```

```
plt.ylabel("Cumulative Reward")
```

```
plt.title("Cumulative Reward per Episode")
```

```
plt.grid(True)
```

```
plt.show()
```

Step 3: Convergence Analysis

During the first few episodes, the agent performs more exploration because ϵ is initially high.

As the number of episodes increases:

1. The agent interacts repeatedly with the environment.
2. Q-values are updated.
3. Good actions receive higher Q-values.
4. Obstacle actions receive negative rewards.
5. Epsilon decreases gradually.
6. Random exploration decreases.
7. The agent starts exploiting learned actions.
8. The policy becomes more stable.

Therefore, the stabilization of Q-values and repeated selection of useful actions indicate that the

Q-Learning algorithm is approaching convergence.

The reward curve may fluctuate because the agent continues to explore during learning.

However, the overall learning behaviour improves as the agent discovers a route toward the goal.

Result

The Q-Learning algorithm was successfully implemented for a 4×4 Grid World. The environment contains 16 states, four possible actions, obstacles, and a goal state.

The Q-table was initialized with zeros and updated for 100 episodes. Q-values were recorded at Episodes 1, 50, and 100. The final learned policy was extracted using the maximum Q-value for each state.

Conclusion

The experiment demonstrates that Q-Learning enables an agent to learn an effective policy through trial and error. The agent gradually learns to avoid obstacles and move toward the goal. The improvement and stabilization of Q-values demonstrate the learning and convergence of the algorithm.